GoalProof

Solana devnet

Prediction markets that settle themselves.

Micro-markets on live World Cup stats. When a period ends, the Solana program re-verifies a **TxLINE Merkle proof on-chain** and pays out only if the math holds — no admin, no trusted oracle, no human in the loop.

✓ real on-chain settlement — every tx on Explorer

goalproof-chi.vercel.app

THE PROBLEM

Every prediction market has the same weak link: the oracle

Markets live on-chain, but the *outcome* almost never does. Someone you must trust types in the result.

Status quo — reported settlement

- ✕ An admin key or oracle service declares the winner off-chain.
- ✕ You trust that they read the feed correctly — and honestly.
- ✕ Disputes are a support ticket, not a proof.
- ✕ One compromised reporter = every open market at risk.

GoalProof — verified settlement

- ✓ TxLINE anchors match stats into a daily Merkle root **on Solana** .
- ✓ The program re-checks the stat's Merkle proof **on-chain via CPI** .
- ✓ Payout executes only if the proof + predicate verify.
- ✓ There is **no manual override anywhere in the program** .

THE PRODUCT

A quiz for every minute you watch

For each fixture, GoalProof auto-generates a pack of 7 provable micro-markets — only stats TxLINE can Merkle-prove are ever listed.

7 markets

PER FIXTURE, AUTO-GENERATED

- Total goals > 2 · match winner
- Red card · corners > 9 · yellows > 4
- Goal in 1st half · goal in 2nd half

Live board

MARKETS FOLLOW THE MATCH

- OPEN → LOCKED at kick-off
- Proofs fetched at HT & FT
- SETTLED with explorer receipts

Match Theater

CSS-3D PITCH, FEED-DRIVEN

- Goals, cards & corners land live on a 3D pitch
- Mouse-driven camera, zero dependencies
- Citron beam = the on-chain proof moment

Replay mode ships in the product: the judged demo replays fixture 18213979 (Norway 1–2 England) deterministically — required, since World Cup fixtures end before judging.

SETTLEMENT PIPELINE

From whistle to payout, in three moves

01

Anchored

TxLINE commits every match stat into a **daily Merkle root stored on Solana**. The truth is on-chain before any market resolves.

→

02

Proven

At period end, the keeper bot pulls that stat's Merkle proof from `/scores/stat-validation` and submits it to the program.

→

03

Verified

`resolve` CPIs into txoracle `validate_stat`: proof re-hashed to the root, predicate evaluated, **payout only if it verifies**.

Deterministic by construction: the outcome is **derived from the proven value**, never asserted. The NO side settles on the exact **complement predicate** — both outcomes equally proof-backed.

NOT A MOCKUP

One market, start to finish, on devnet

"Norway vs England: total goals > 2?" — opened, bet on both sides, resolved by Merkle proof, paid out.
Every signature resolves on Solana Explorer.

3 goals
PROVEN STAT (1 + 2)

YES · paid
OUTCOME

1.4M CU
COMPUTE FOR MERKLE VERIFY

0 humans
IN THE SETTLEMENT LOOP

program	8RnQwJk6FN5rioUaGqEruyXENxQZNBdgJQUHCgpr4MNP
market	EzFT9cpfKBpaL43TTJQb1u3k7hC1oNdtMaG2ydhTUSDB
create	7quy5n…L5nBf
bet YES / bet NO	4S2G4N…Y8akgp · 7EAY3v…LPwvr5
resolve (CPI)	UqgCj5GxGTAmCxTJ89yD7pfhs7axp8NBdn…bQNMMy
claim payout	5RajCEi3LAzdA6NC5zYWUaCpaKNLhyibxS…mzrR9X
merkle root (event)	011d84f6c3121bb7…2b2cabec

Tx logs show: `goalproof → CPI txoracle::validate_stat → "Find valid on-chain root" → predicate ✓ → success`

ARCHITECTURE

Four services, one network

Everything shares one devnet — same RPC, program ID, SPL mint, and TxLINE endpoint. Clean module boundaries; the dashboard reads one `MarketState` contract.

ingestion/ — Go

Consumes TxLINE SSE (`scores` + `odds`), records JSONL → deterministic replay.

program/ — Anchor · Rust

`create_market` · `take_position` · `resolve` (CPI `validate_stat`) · `claim` — escrow in a program vault.

keeper/ — TypeScript

Watches the feed, detects period end, fetches the proof, submits `resolve`. No human trigger.

web/ — React · Vite

Live board + Match Theater + replay scrubber; settled cards link to the real devnet txs.

```
feed → ingestion → recordings / live → keeper → proof → program (verify on-chain) → payout → web receipts
```

CODE QUALITY · JUDGED CRITERION

Three findings that made the CPI actually work

The docs demonstrate `validate_stat` as an off-chain `.view()`. Making it a real on-chain CPI took verified engineering:

32-byte array hashes

TxLINE proof hashes must be passed as raw `[u8; 32]` arrays, not hex strings — the API returns hex, the keeper decodes before building the instruction.

Roots-PDA derivation

The scores-roots account is a PDA of `[b"daily_scores_roots", u16(epochDay)]` — undocumented; recovered by scanning devnet transactions.

1.4M compute units

Merkle verification blows the default budget; `resolve` ships with a `ComputeBudget` instruction raising it to 1.4M CU.

— Deterministic resolution code — outcome derived from the proven value; NO settled via exact predicate complement.

— Five documented verification points in the settlement path; market catalog shared 1:1 between keeper and dashboard.

— Honest UI: marker positions labeled illustrative (TxLINE has no player tracking); mock vs real settlement clearly separated in code.

TXLINE INTEGRATION

Endpoints used & feedback logged from day one

Endpoints

auth	POST /auth/guest/start · token activate
live streams	GET /odds/stream · /scores/stream (SSE)
proofs	GET /scores/stat-validation
on-chain	txoracle 6pW64g…yP2J (devnet)
predicates	2-stat op (add/sub) + threshold + comparison

API feedback (docs/txline-feedback-log.md)

- CPI usage of `validate_stat` undocumented — confirmed viable (1 read-only account, no signer); worth an official example.
- Proof hash encoding (hex vs bytes) and roots-PDA seeds had to be reverse-engineered.
- Guest JWT flow is smooth; SSE streams stable through full match recordings.
- Compliance: no TxL token wagering — escrow is our own devnet SPL mint.

Full friction log with dates and workarounds ships in the repo as part of the submission.

DEMO · ≤ 5 MINUTES

What you'll see in the video

- **Replay a full match** — Norway 1–2 England from a real recorded TxLINE feed; the Match Theater comes alive with goals, cards and corners.
- **Markets live their lifecycle** — OPEN → LOCKED → SETTLED as events land; proofs fetched at HT and FT (citron beam sweep).
- **The money shot** — the settled "total goals > 2" card links to the real `resolve` tx: CPI into `validate_stat`, Merkle root found, predicate verified, payout claimed.
- **Architecture in 50 seconds** — the four services and the deterministic settlement path.

Try it yourself

live site	goalproof-chi.vercel.app
live board	…/#/live → press ▶
program	8RnQwJ…4MNP on devnet
repo	public GitHub · MIT

Provable markets, settled trustlessly.

Every settlement is independently verifiable on Solana Explorer.
The resolution code is the spec — clean, deterministic, and open.

goalproof-chi.vercel.app

resolve tx: UqgCj5…QNMMy

devnet · MIT